Übungsblatt 2 — Bäume (2) Lösung

Aufgabe 1: AVL-Bäume – Einfügen und Löschen

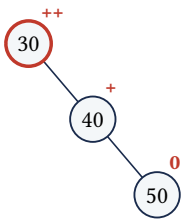
Gegeben sei ein leerer AVL-Baum.

1. Konstruieren Sie den AVL-Baum durch Einfügen der Schlüssel in der Reihenfolge: `30, 40, 50, 20, 25`

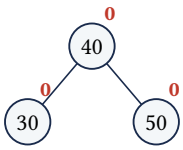
Füge solange ein, bis eine Ausgleichsmaßnahme nötig ist.

Einfügen von 30: passt.
Einfügen von 40: passt.
Einfügen von 50: Ungleichgewicht bei 30 (++), Fall rechts-rechts: Einfachrotation L(40).
Einfügen von 20: passt.
Einfügen von 25: Ungleichgewicht bei 30 (−−), Fall links-rechts: Doppelrotation LR(25).

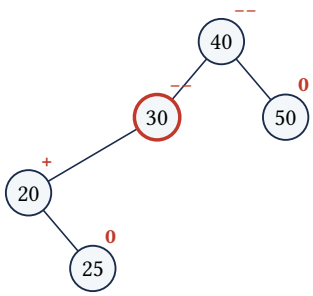
Einfügen 50: Ungleichgewicht bei 30 (++)



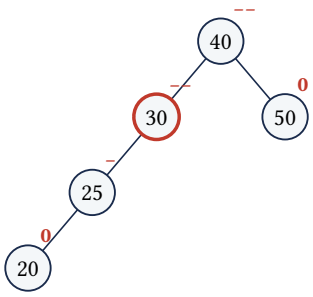
Nach L(40)



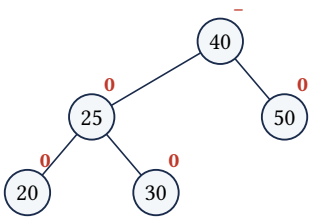
Einfügen 25: Ungleichgewicht bei 30 (−−)



Zwischenschritt: 1. L(25)



Nach LR(25)

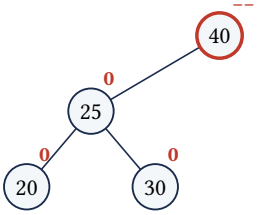


2. Löschen Sie den Knoten `50` und anschließend den Knoten `25`.

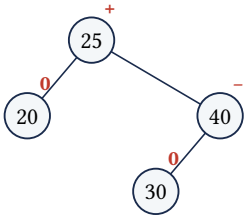
Löschen kann ggf. auch zu mehreren kaskadierenden Ausgleichsmaßnahmen führen. Hier nicht nötig.

Löschen von 50: Ungleichgewicht bei 40 (Balancefaktor −−), Fall linkes Kind ausgeglichen (Sonderfall beim Löschen): Einfachrotation R(25).
Löschen von 25: Der Knoten hat zwei Kinder und wird durch den symmetrischen Vorgänger 20 ersetzt. Ungleichgewicht bei 20 (Balancefaktor ++), Fall rechts-links: Doppelrotation RL(30) (1. R(30), 2. L(30)).

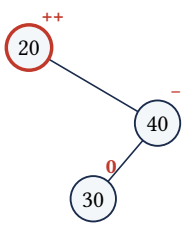
Löschen 50: Ungleichgewicht bei 40 (−−)



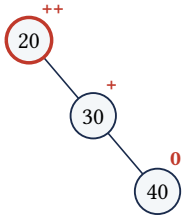
Nach R(25)



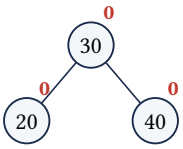
Löschen 25: Ungleichgewicht bei 20 (++)



Zwischenschritt: 1. R(30)



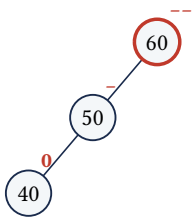
Nach RL(30)



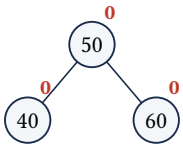
3. **Optional**: Gegeben sei erneut ein leerer AVL-Baum. Fügen Sie die Schlüssel `60, 50, 40, 70, 65` ein.

Einfügen von 60: Die AVL-Bedingung ist weiterhin erfüllt, keine Ausgleichsmaßnahme nötig.
Einfügen von 50: Die AVL-Bedingung ist weiterhin erfüllt, keine Ausgleichsmaßnahme nötig.
Einfügen von 40: Ungleichgewicht bei 60 (Balancefaktor −−), Fall links-links: Einfachrotation R(50).
Einfügen von 70: Die AVL-Bedingung ist weiterhin erfüllt, keine Ausgleichsmaßnahme nötig.
Einfügen von 65: Ungleichgewicht bei 60 (Balancefaktor ++), Fall rechts-links: Doppelrotation RL(65) (1. R(65), 2. L(65)).

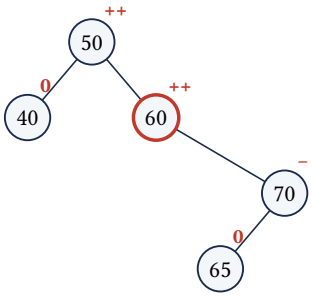
Einfügen 40: Ungleichgewicht bei 60 (−−)



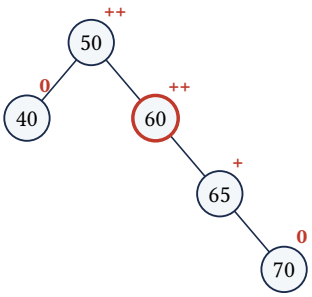
Nach R(50)



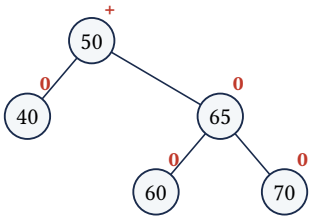
Einfügen 65: Ungleichgewicht bei 60 (++)



Zwischenschritt: 1. R(65)



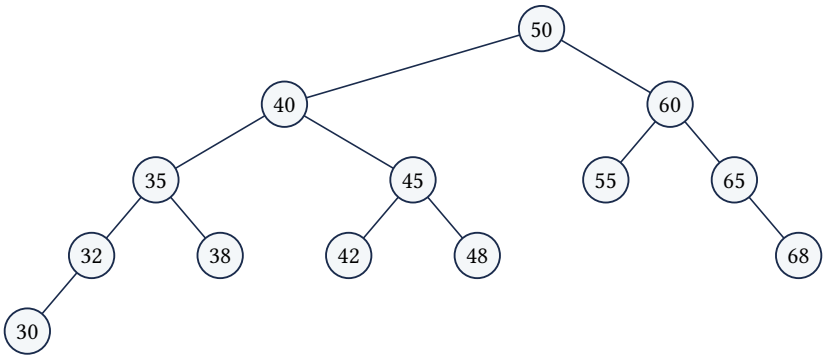
Nach RL(65)



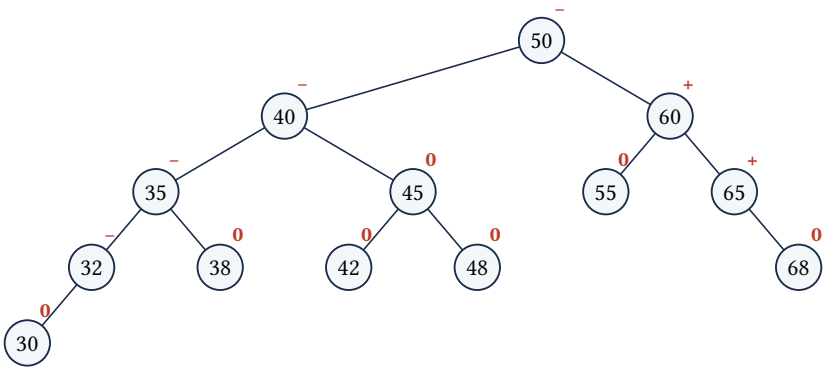
> **Hinweis:** Betrachten Sie das Einfügen und Löschen jedes Schlüssels als eigene Operation und prüfen Sie danach die AVL-Bedingung. Benennen Sie jede Ausgleichsmaßnahme (z. B. L(40)).

Aufgabe 2: AVL-Bäume – Löschen mit Kaskade

Gegeben sei der folgende AVL-Baum.



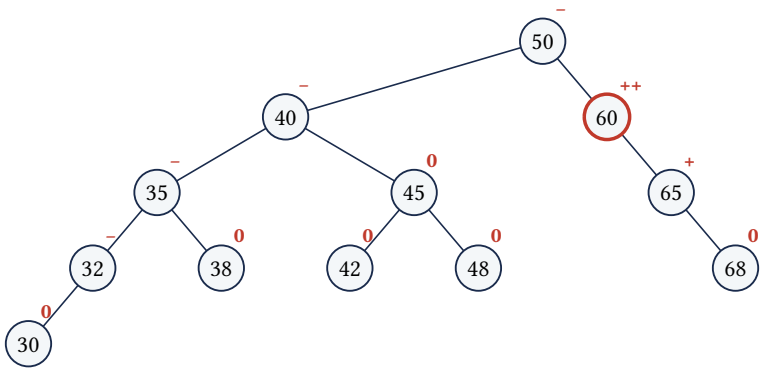
1. Ergänzen Sie die Balancefaktoren der Knoten.



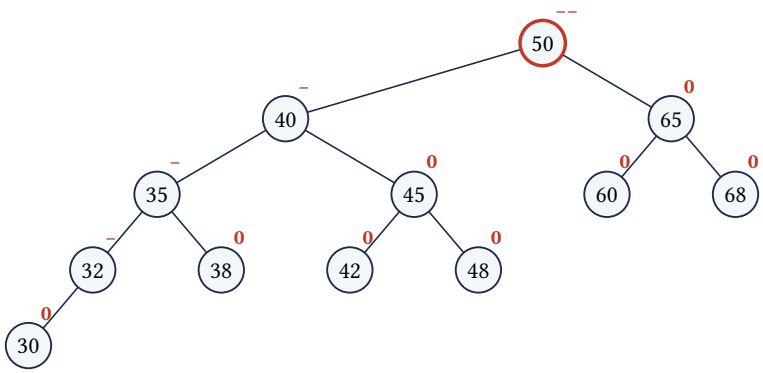
2. Löschen Sie den Knoten 55.

Löschen von 55: Ungleichgewicht bei 60 (++), Fall rechts-rechts: Einfachrotation L(65).
Ungleichgewicht bei 50 (––), Fall links-links: Einfachrotation R(40).

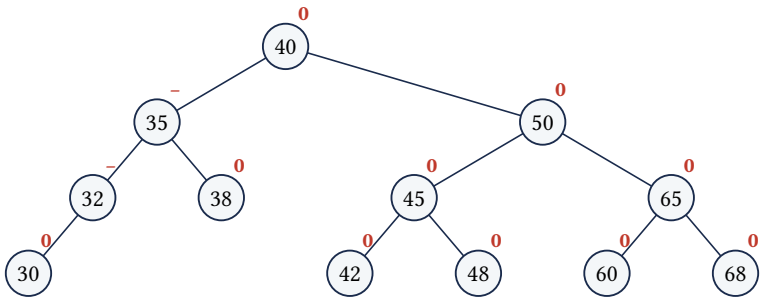
Löschen 55: Ungleichgewicht bei 60 (++)



Löschen 55: Ungleichgewicht bei 50 (−−)



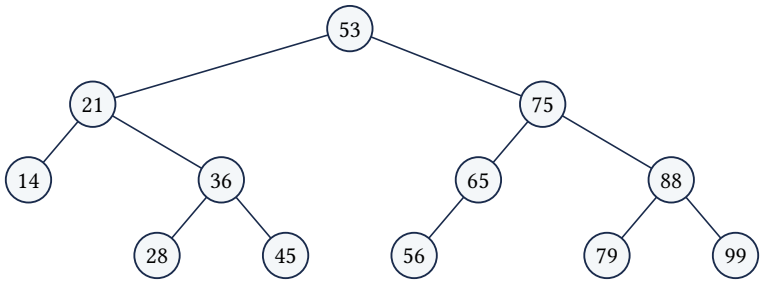
Nach R(40)



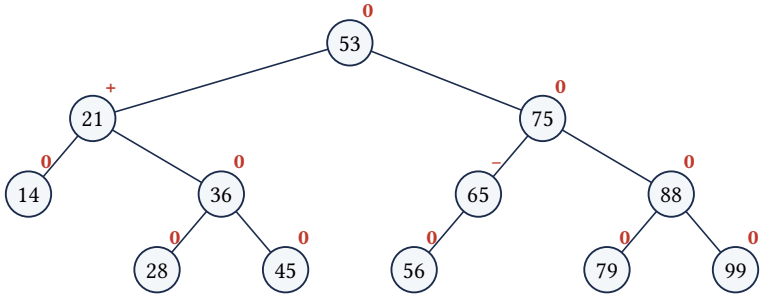
Hinweis: Löschen kann zu kaskadierenden Ausgleichsmaßnahmen führen: Nach einer Rotation kann sich die Höhe des Teilbaums verringert haben, dann muss der Weg bis zur Wurzel weiter geprüft werden.

Aufgabe 3: AVL-Bäume - Löschen (weitere Fälle)

Gegeben sei der folgende AVL-Baum.



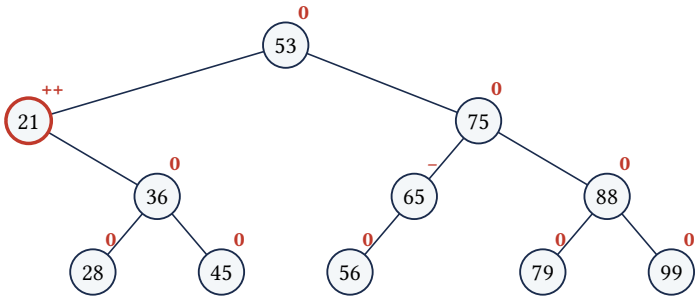
1. Ergänzen Sie die Balancefaktoren der Knoten.



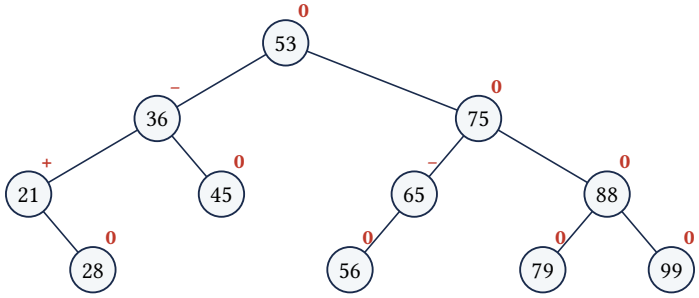
2. Löschen Sie nacheinander die Knoten `14`, `45` und `28`. Geben Sie jeweils an, ob und welche Ausgleichsmaßnahme nötig ist.

Löschen von 14: Ungleichgewicht bei 21 (++), Fall rechtes Kind ausgeglichen: Einfachrotation L(36).
Löschen von 45: Ungleichgewicht bei 36 (−−), Fall links-rechts: Doppelrotation LR(28).
Löschen von 28: Der Knoten hat zwei Kinder und wird durch den symmetrischen Vorgänger 21 ersetzt. Die AVL-Bedingung ist weiterhin erfüllt, keine Ausgleichsmaßnahme nötig.

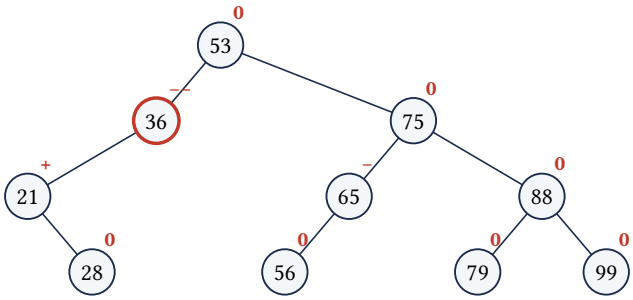
Löschen 14: Ungleichgewicht bei 21 (++)



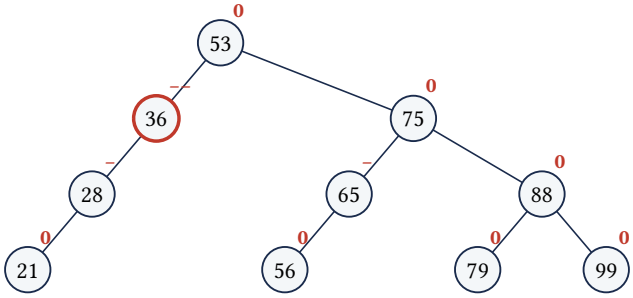
Nach L(36)



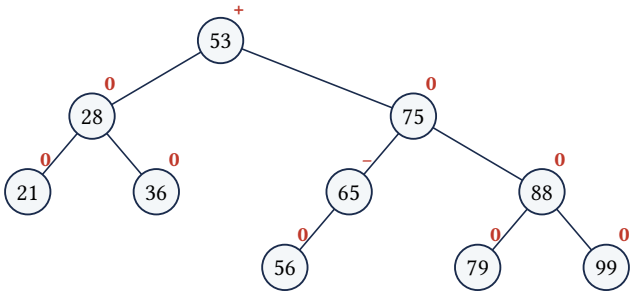
Löschen 45: Ungleichgewicht bei 36 (−−)



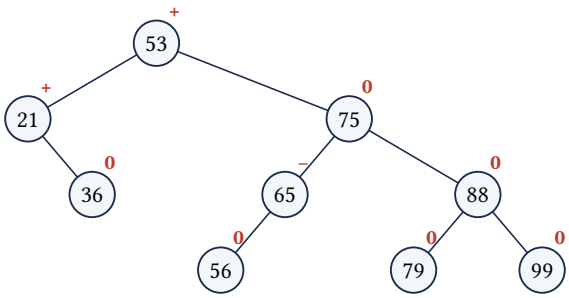
Zwischenschritt: 1. L(28)



Nach LR(28)

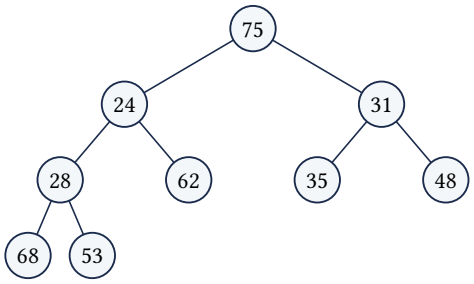


Ergebnis



Aufgabe 4: Heaps korrigieren

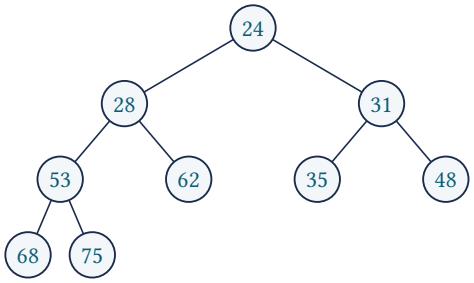
Gegeben seien der folgenden fast vollständigen (Min-)Heap, in dem genau ein Knoten falsch ist.



Ermitteln Sie den falschen Knoten und korrigieren Sie den Heap durch Anwendung eines bekannten Algorithmus.

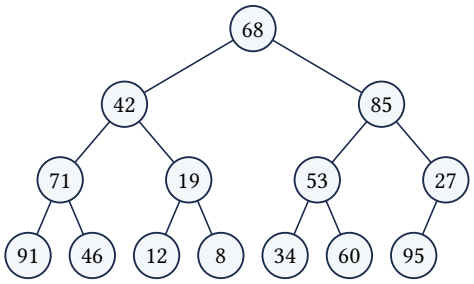
Die Wurzel 75 ist falsch, sie ist größer als ihre Kinder.

DownHeap: Falschen Knoten immer mit dem kleineren Kind tauschen. 75 ↔ 24, 75 ↔ 28, 75 ↔ 53, 75 ist Blatt.



Aufgabe 5: Heaps – BuildHeap

Gegeben sei der folgende binäre Baum.



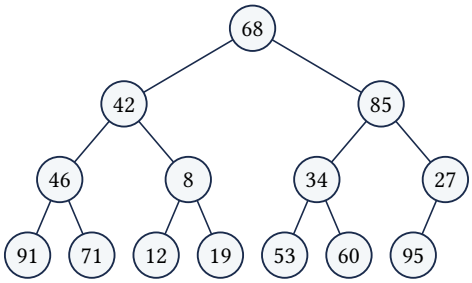
1. Erklären Sie kurz in eigenen Worten den Algorithmus `BuildHeap`.

BuildHeap erzeugt aus einem beliebigen binären Baum einen gültigen Heap. Er durchläuft alle Knoten von unten nach oben und von rechts nach links, beginnend beim letzten Knoten mit mindestens einem Kind. Für jeden Knoten wird der Algorithmus DownHeap ausgeführt. Dieser lässt den Knoten soweit einsickern, bis die Heap-Bedingung für den Teilbaum erfüllt ist. Blätter erfüllen die Heap-Bedingung immer und werden übersprungen.

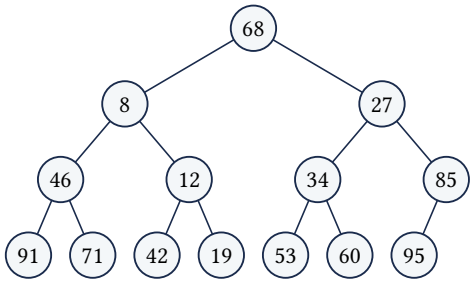
2. Erzeugen Sie aus dem gegebenen Baum unter Anwendung bekannter Algorithmen einen Heap.

DownHeap(27): kein Tausch
DownHeap(53): 53 ↔ 34
DownHeap(19): 19 ↔ 8
DownHeap(71): 71 ↔ 46
DownHeap(85): 85 ↔ 27
DownHeap(42): 42 ↔ 8, 42 ↔ 12
DownHeap(68): 68 ↔ 8, 68 ↔ 12, 68 ↔ 19

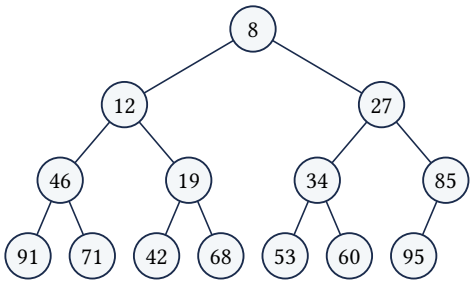
Nach DownHeap auf der vorletzten Ebene



Nach DownHeap auf Ebene 1



Ergebnis nach DownHeap der Wurzel



> **Hinweis:** Beim Angeben von Zwischenergebnissen reicht die Kurzschreibweise.

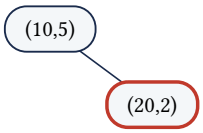
Aufgabe 6: Treaps – Einfügen und Löschen

Gegeben sei ein leerer Treap.

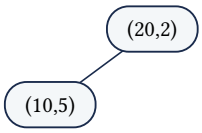
1. Fügen Sie die folgenden Schlüssel-Prioritäts-Paare in der gegebenen Reihenfolge ein: `(10,5), (20,2), (5,7), (15,1), (3,9), (25,4), (12,3)`

Einfügen von (10,5): Der Baum ist leer, das Element wird Wurzel.
Einfügen von (20,2): Als Blatt einfügen, die Priorität ist kleiner als die des Elternknotens, daher rotieren bis die Heap-Bedingung erfüllt ist: L((20,2)).
Einfügen von (5,7): Als Blatt einfügen, die Priorität ist nicht kleiner als die des Elternknotens, keine Rotation.
Einfügen von (15,1): Als Blatt einfügen, die Priorität ist kleiner als die des Elternknotens, daher rotieren bis die Heap-Bedingung erfüllt ist: L((15,1)), R((15,1)).
Einfügen von (3,9): Als Blatt einfügen, die Priorität ist nicht kleiner als die des Elternknotens, keine Rotation.
Einfügen von (25,4): Als Blatt einfügen, die Priorität ist nicht kleiner als die des Elternknotens, keine Rotation.
Einfügen von (12,3): Als Blatt einfügen, die Priorität ist kleiner als die des Elternknotens, daher rotieren bis die Heap-Bedingung erfüllt ist: L((12,3)).

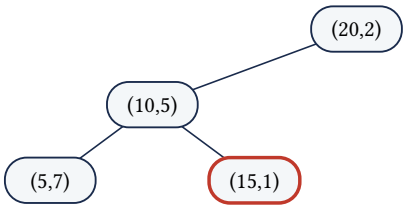
Einfügen (20,2) als Blatt



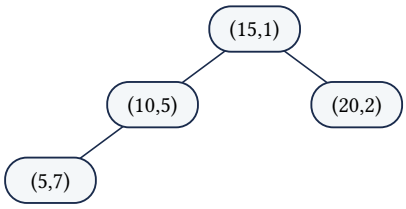
Nach L((20,2))



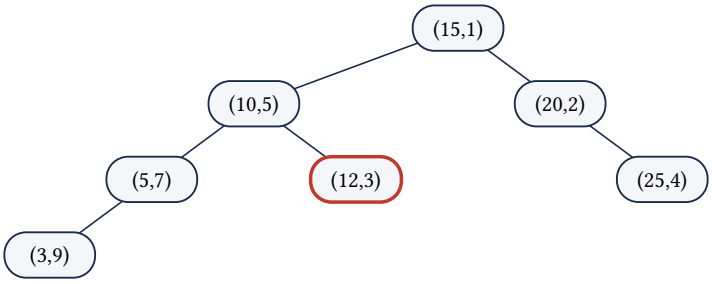
Einfügen (15,1) als Blatt



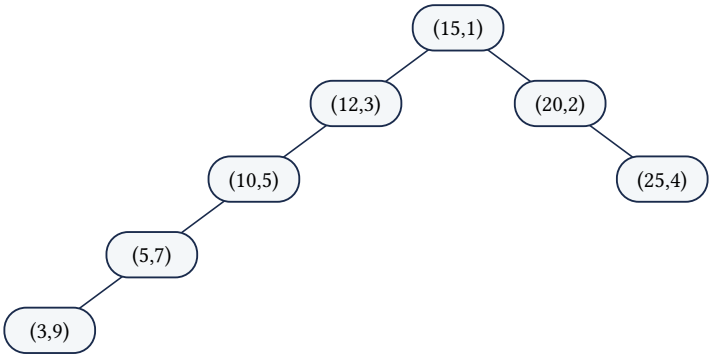
Nach L((15,1)), R((15,1))



Einfügen (12,3) als Blatt



Nach L((12,3))

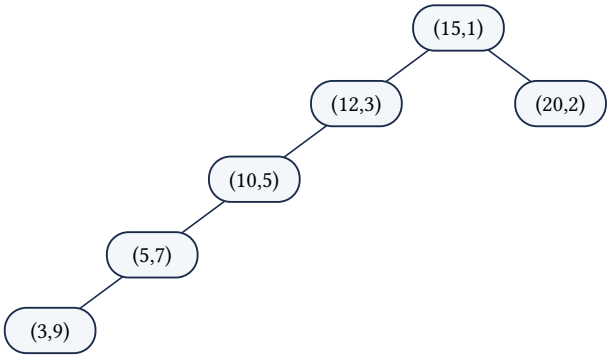


2. Löschen Sie nacheinander die Elemente `(25,4)`, `(10,5)` und `(15,1)` aus dem bestehenden Treap.

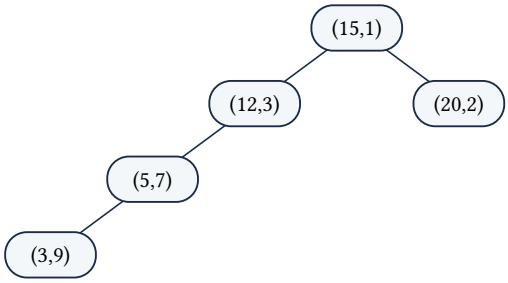
Löschen von (25,4): Blätter werden einfach ausgehängt.
Löschen von (10,5): Kein Blatt ⟶ rotieren bis Blatt: R((5,7)) ⟶ Blatt aushängen.

Löschen von (15,1): Kein Blatt → rotieren bis Blatt (Kind mit kleiner Prio nach oben): L((20,2)), R((12,3)) → Blatt aushängen.

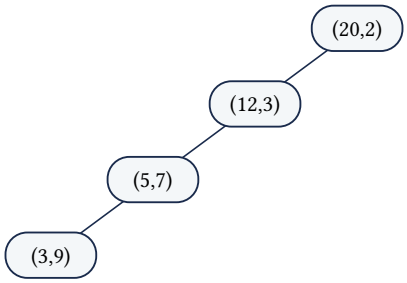
Nach Löschen von (25,4)



Nach Löschen von (10,5)



Nach Löschen von (15,1)



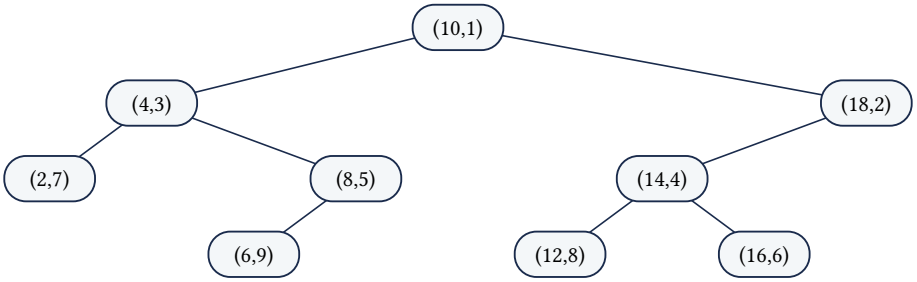
Hinweis: Notation L((k,p)) bzw. R((k,p)) ist die Links- bzw. Rechtsrotation. k nach Suchbaum, p nach Min-Heap.

Aufgabe 7: Treaps – Eindeutigkeit und Rotationen

Gegeben sei die Menge von Schlüssel-Prioritäts-Paaren:
`S = {(2,7), (4,3), (6,9), (8,5), (10,1), (12,8), (14,4), (16,6), (18,2)}`.

1. Konstruieren Sie den Treap für S direkt, keine Zwischenschritte nötig.

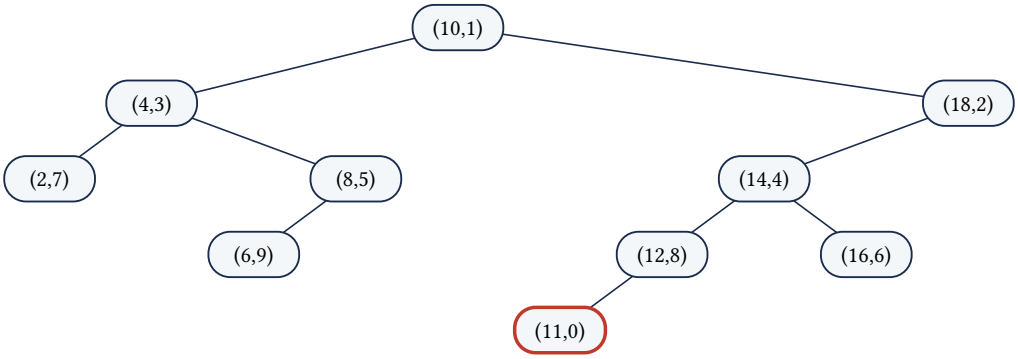
Die Wurzel ist das Element mit der kleinsten Priorität: (10,1). Alle Schlüssel kleiner als 10 liegen im linken, alle größeren im rechten Teilbaum. Dort gilt rekursiv dasselbe: links ist (4,3) die Wurzel, rechts (18,2).



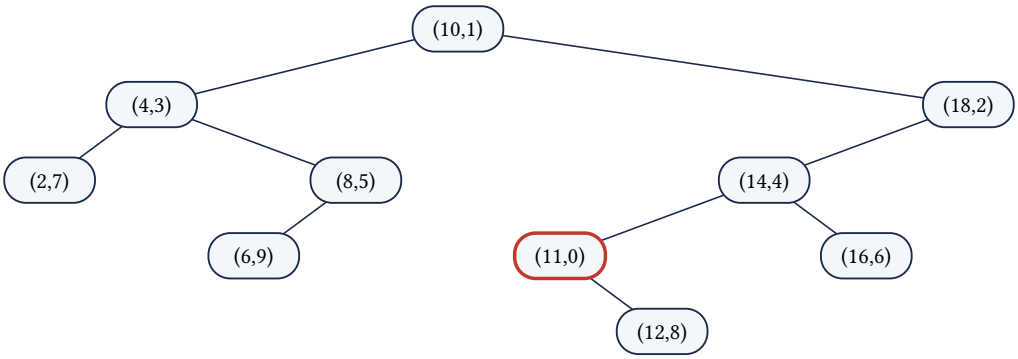
2. Fügen Sie in den Treap aus Teil 1 das Element `(11,0)` ein und illustrieren Sie alle Zwischenschritte.

Rotationen: R((11,0)), R((11,0)), R((11,0)), L((11,0)). Das Element (11,0) hat die kleinste Priorität und wandert deshalb bis zur Wurzel.

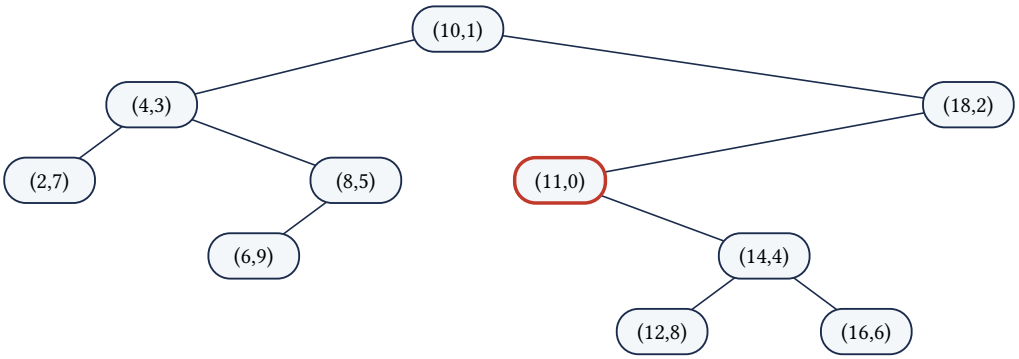
(11,0) als Blatt eingefügt



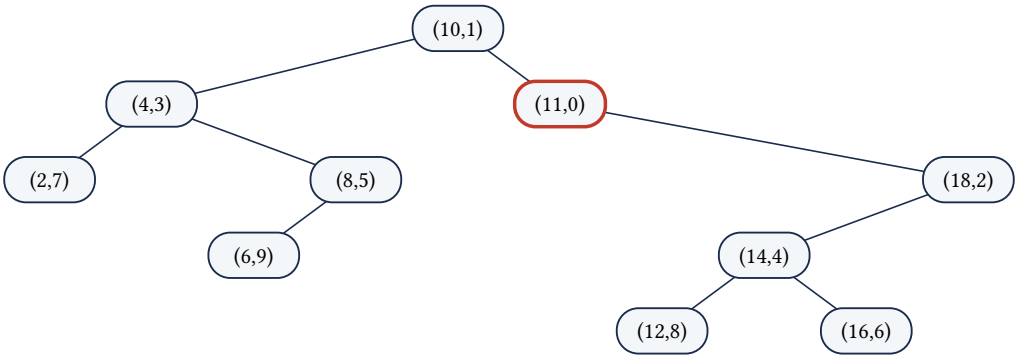
Nach R((11,0))



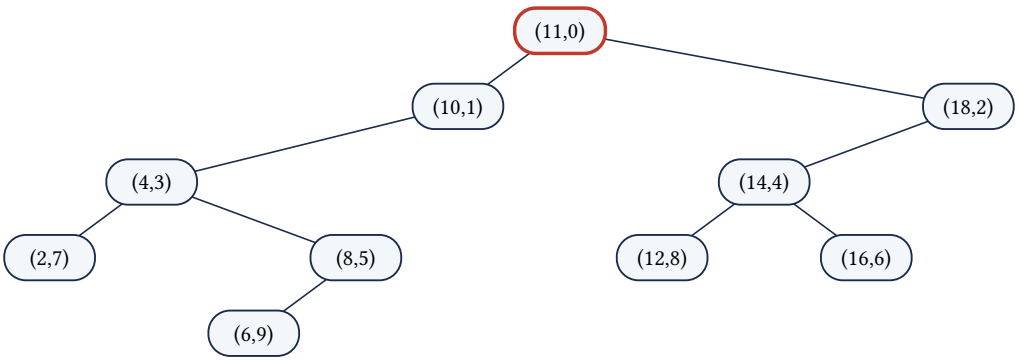
Nach R((11,0))



Nach R((11,0))

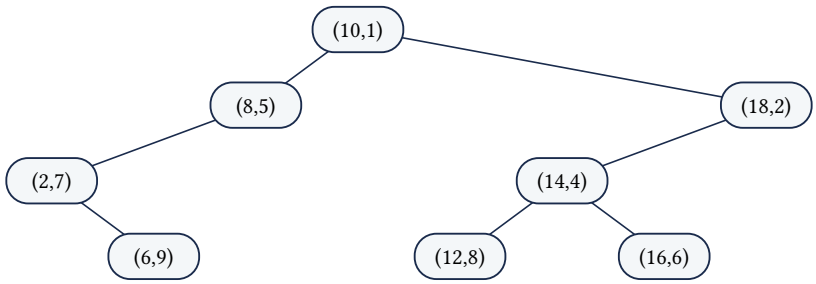


Nach L((11,0))



3. **Optional**: Löschen Sie aus dem Treap aus Teil 1 das Element `(4,3)`.

Das Element (4,3) ist kein Blatt: Rotieren (das Kind mit der kleineren Priorität rückt nach oben), bis es ein Blatt ist: L((8,5)), R((2,7)), L((6,9)). Dann aushängen.

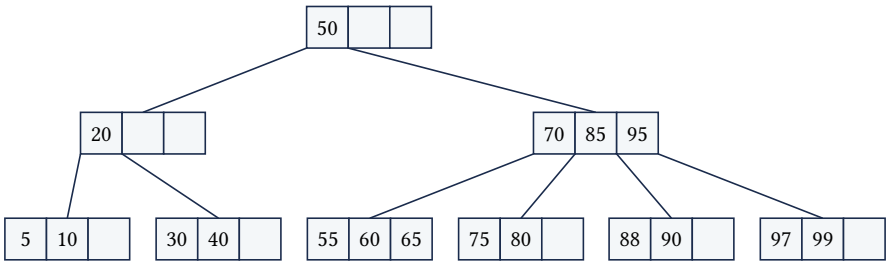


4. Begründen Sie, warum der Treap bei gegebenen Schlüssel-Prioritäts-Paaren (bei paarweise verschiedenen Schlüsseln und Prioritäten) unabhängig von der Einfügereihenfolge immer derselbe ist.

Ein Treap ist gleichzeitig Suchbaum bezüglich der Schlüssel und Heap bezüglich der Prioritäten. Im Heap muss die Wurzel das Element mit der kleinsten Priorität sein, also ist die Wurzel eindeutig festgelegt. Wegen der Suchbaum-Eigenschaft sind dadurch auch die Mengen für den linken und rechten Teilbaum eindeutig festgelegt. Für diese Teilbäume gilt dasselbe, also ist die gesamte Struktur eindeutig. Die Einfügereihenfolge und die dabei nötigen Rotationen haben darauf keinen Einfluss.

Aufgabe 8: B-Bäume – Ordnung und Eigenschaften

Gegeben sei der folgende B-Baum.



1. Welche Ordnung d hat dieser B-Baum? Wie viele Elemente muss eine Seite mindestens und höchstens enthalten (Wurzel und übrige Seiten)? Wie viele Nachfolger kann eine innere Seite haben?

Jede Seite hat Platz für 3 Elemente und 4 Zeiger $\Longrightarrow$ d = 4.

Jeder innere Knoten und jedes Blatt hat mindestens $\left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{4-1}{2} \right\rfloor = 1$ Element.
Jeder innere Knoten und jedes Blatt hat maximal $d - 1 = 4 - 1 = 3$ Elemente.
Die Wurzel hat mindestens 1 Element.
Die Wurzel hat maximal $d - 1 = 4 - 1 = 3$ Elemente.

Jeder innere Knoten hat minimal $\left\lceil \frac{d}{2} \right\rceil = \left\lceil \frac{4}{2} \right\rceil = 2$ Nachfolger.
Die Wurzel hat mindestens 2 Nachfolger.
Die Wurzel und jeder innere Knoten hat maximal $d = 4$ Nachfolger.
Jedes Blatt hat 0 Nachfolger.

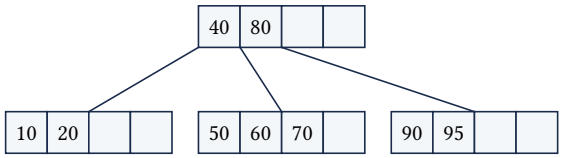
2. Verfolgen Sie die Suche (`BTreeSearch`) nach den Elementen `80` und `62`. Geben Sie jeweils die besuchten Seiten und das Ergebnis an. Was fällt bei der erfolglosen Suche auf?

Suche nach 80:
Nicht in Wurzelseite enthalten $80 > 50 \Longrightarrow$ rechts von 80 weitersuchen.
In `[70,85,95]` ist 80 nicht enthalten und $70 < 80 < 85 \Longrightarrow$ zwischen 70 und 85 weitersuchen.
In `[75,80]` an Indexposition 2 enthalten.

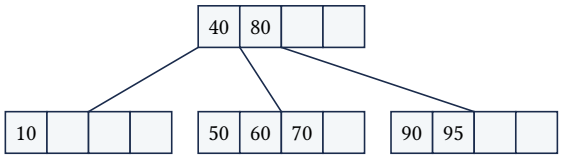
Suche nach 62:
`[50]` $\Longrightarrow$ `[20]` $\Longrightarrow$ `[30,40]` $\Longrightarrow$ Element nicht enthalten und Seite ist Blatt.
Erfolglose Suche endet und liefert $(p, 0), p = [55, 60, 65]$. Falls gesuchtes Element nicht vorhanden ist, wird das Blatt geliefert in das das Element eingefügt werden würde.

Betrachten Sie nun die folgenden Bäume.

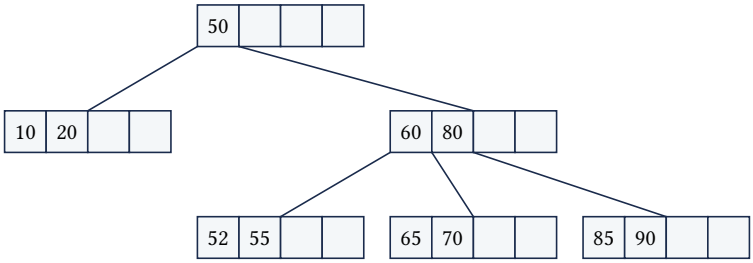
a)



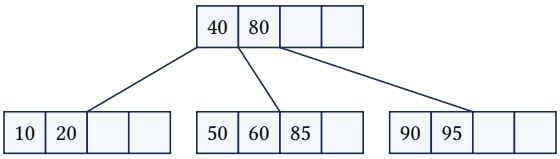
b)



c)



d)



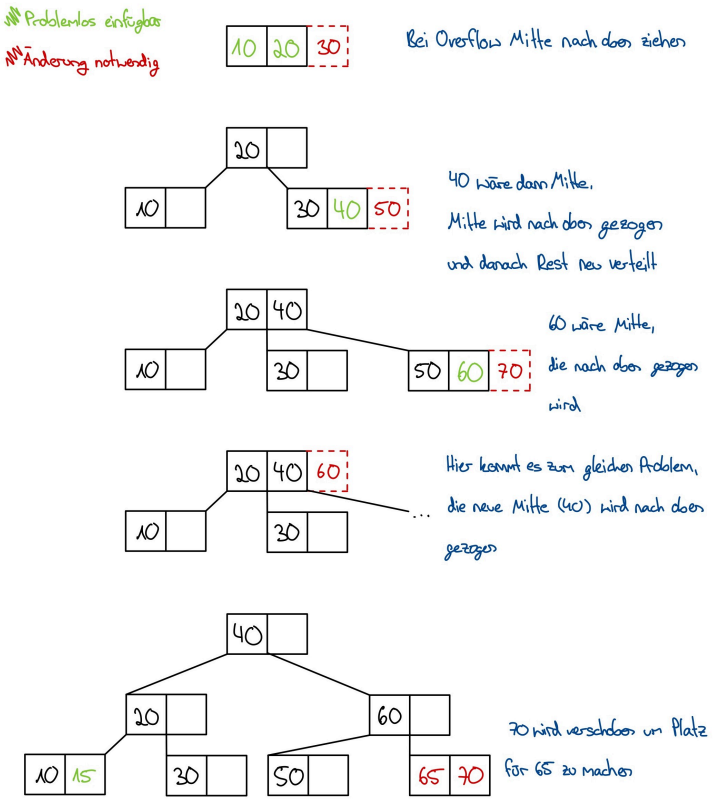
3. Entscheiden Sie für jeden der Bäume a) bis e), ob er ein B-Baum der Ordnung 5 ist. Falls nicht, nennen Sie die verletzte Eigenschaft.

Für $d = 5$ gilt: Höchstens 4 Elemente je Seite, außer der Wurzel mindestens $\left\lfloor \frac{5-1}{2} \right\rfloor = 2$ Elemente, alle Blätter auf einer Ebene, Elemente sortiert und durch die Elemente der Vaterseite eingegrenzt.
a) Ja, alle Eigenschaften sind erfüllt (die Wurzel darf nur 1 Element enthalten).
b) Nein. Das Blatt `[10]` enthält nur 1 Element, mindestens 2 sind nötig (Mindestbelegung).
c) Nein. Das linke Blatt `[10, 20]` liegt auf Ebene 1, die übrigen Blätter auf Ebene 2: Nicht alle Blätter liegen auf derselben Ebene.
d) Nein. Die Seite `[50, 60, 85]` liegt zwischen 40 und 80, enthält aber 85. Damit ist die Suchbaumeigenschaft verletzt (85 gehört in die Seite rechts von 80).

Aufgabe 9: B-Bäume – Einfügen & Löschen

Gegeben sei ein leerer B-Baum der Ordnung 1.

1. Fügen Sie die folgenden Schlüssel in der gegebenen Reihenfolge ein: `10, 20, 30, 40, 50, 60, 70, 15, 65`



2. Löschen Sie nacheinander die Schlüssel `15, 20, 50` aus dem bestehenden B-Baum.

